

PROJECT REPORT ON

SIMPLE STUDENT MANAGEMENT SYSTEM

Course: Programming in Java

Document: Report

Date: September 2026

Language : Java

Storage : student_records.txt

Introduction-

The Simple Student Management System is a Java program designed to illustrate fundamental concepts of software structure and makes use of Object-Oriented Programming, the Java Collection Frameworks, and File I/O stream operations in order to provide a means of managing records for different student groups.

The project illustrates how to use inheritance, how to override methods, how to handle collections using ArrayList, how to validate the input with the Java library functions, and how to save the data to files.

Problem Statement

It is common for schools and basic software systems to keep track of students, and simple command line tools have two principal problems:

- 1 Unless it is saved, any data that is added during the operation of the program is lost when the program ends.

2. Simple systems make use of fixed arrays and this means that they are not well suited to dealing with different types of students, which in turn makes the code difficult to alter.

It deals with this by providing a method of saving data and by using a class structure.

FUNCTIONAL REQUIREMENT

- 1) View Records | The system goes through the list of students and displays the details for each one in a variety of ways.
- 2) To add a student | The programme obtains from the user details such as name and age, creates a student and then adds them to the list.
- 3) Save and Exit | Stores all the student information in a file and terminates the program.
- 4) Regarding input handling | The program will not crash if the user enters the wrong kind of information.

Non-Functional Requirements

The system has to continue operating even if errors occur.

The code should be easy to modify in order to add different types of students.

The program will function on any system that has Java 8 or a later version.

The program has to be fast if it is to be used.

. System Architecture

The application features a three-tiered console architecture with the display and the main code being kept separate.

```
graph TD
    subgraph Presentation_Layer [Presentation Layer]
        CLI[Terminal / User Console]
        ScannerIO[Java Scanner Reader]
    end

    Controller[SimpleStudentSystem. Main Controller Loop]

    subgraph Storage_Layer [Storage Layer]
        Collection[ArrayList Memory Storage]
        BaseEntity[Student Base Class]
        DerivedEntity[GradStudent Class]
    end

    CLI <--> ScannerIO
    ScannerIO <--> Controller
    Controller <--> Collection
    Collection --> BaseEntity
    BaseEntity <|-- DerivedEntity
    Controller --> StreamWriter
    StreamWriter --> DiskStorage
```

Design Diagrams

Use Case Diagram

User((System Administrator))

UC1(View All Students)

UC2(Add Standard Student)

UC3(Persist Data & Exit)

User --> UC1

User --> UC2

User --> UC3

WORKFLOW DIAGRAM

Start([Start Program]) --> Init[Initialize ArrayList & Default Records]

Init --> Menu[Display Console Menu Options]

Menu --> Input[Prompt for Choice]

Input --> Validate{Valid Integer?}

Validate -- No --> Error[Display Parse Error] --> Menu

Validate -- Yes --> Choice{Evaluate Switch}

Choice -- Option 1 --> View[Iterate ArrayList & Call display] --> Menu

Choice -- Option 2 --> Add[Read Name & Age -> Add to List] --> Menu

Choice -- Option 3 --> Save[Write Records to File] --> Exit([Exit Execution])

Choice -- Default --> Invalid[Display Invalid Range Warning] --> Menu

Storage Mapping

```
STUDENT_RECORD {
```

```
    string name PK
```

```
    int age
```

```
    string category
```

```
    string research_area
```

```
}
```

```
FILE_STORE {
```

```
    string file_path PK
```

```
    string line_format
```

```
}
```

```
STUDENT_RECORD ||--o{ FILE_STORE : "written to"
```

```
class Student {  
  
    String name  
  
    int age  
  
    Student(String name, int age)  
  
    display() void  
  
    getName() String  
    getAge() int  
  
}
```

```
class GradStudent {  
  
    String researchArea  
  
    GradStudent(String name, int age, String researchArea)  
  
    display() void  
  
}
```

```
class SimpleStudentSystem {  
  
    main(String[] args)  
  
}
```

Student-- GradStudent : Extends

SimpleStudentSystem..> Student : Manages

Design.

That's why we chose ArrayList — because it can grow as required.

We applied the display method in order to show the different types of students without carrying out any checking of the type.

We employed line by line input in order that the program should not fail to pick up any data.

We chose to use a file rather than a database so that the program won't need any additional software.

Here is the Java code, for the system:

```
import java.io.FileWriter;

import java.io.IOException;

import java.util.ArrayList;

import java.util.Scanner;

// Base Class

class Student {

    protected String name;

    protected int age;

    public Student(String name, int age) {

        this.name = name;

        this.age = age;

    }

    void display() {

        System.out.println("Student Name: " + name + ". Age: " + age);

    }

}
```

Challenges Faced

1. Scanner Buffer Mismatches: The use of `scanner.nextInt()` caused the line-feed delimiters to remain in the input buffer, thereby causing trailing string inputs to be overlooked. This problem was solved by standardising the input readings onto `scanner.nextLine()` and then using `Integer.parseInt()`.
2. Lack of superclass context: In the implementations, casting was needed in order to print the fields of the subclass when referring to the base class. This problem was neatly resolved by directly overriding the `display()` method through the use of a virtual method call.
3. Uncontrolled file streams: There was a risk of file handle leaks when `*FileWriter*` declarations were not checked. Resource leaks could be reduced by wrapping the file streams in try-catch blocks.

Lessons and Main Takeaways

An application of runtime polymorphism has been demonstrated through dynamic method dispatch.

Realised that it was necessary to handle inputs defensively in order to keep the CLI stable when the users were running it.

Gained experience using file output streams (FileWriter)

Future Improvements

File hydration at startup: The program will be modified to read and parse stored records from the file `student_records.txt` during startup.

CRUD Functions: Include features such as searching, record modification and deletion in the main menu loop.

Database Connectivity: Modify the system from saving data in a file to storing them in an embedded database engine such as SQLite or H2 with JDBC connections.

GUI: Replace the command line interface with a Graphical User Interface GUI through JavaFX or Swing.

15. References

Bloch, Joshua. Effective Java (3rd Edition).

Addison-Wesley Professional, 2018.

Schildt, Herbert.

Java: The Complete Reference (12th Edition). McGraw-Hill Education,
2021.